

Instruction Set Architecture

The operation of a processor is determined by the instructions it executes, referred to as *machine instructions* or *computer instructions*. The collection of different instructions that a processor can execute is then called the processor's **instruction set**. Each instruction must contain specific information required by the processor for execution. The following are the elements of machine instruction:

- **Operation code** – This specifies the operation to be performed. It is also known as the opcode.
- **Source operand reference** – This encompasses the input for the operation. An operation may involve one or more source operands.
- **Result operand reference** – This encompasses the results of the operation.
- **Next instruction reference** – This indicates where the processor should fetch the next instruction.

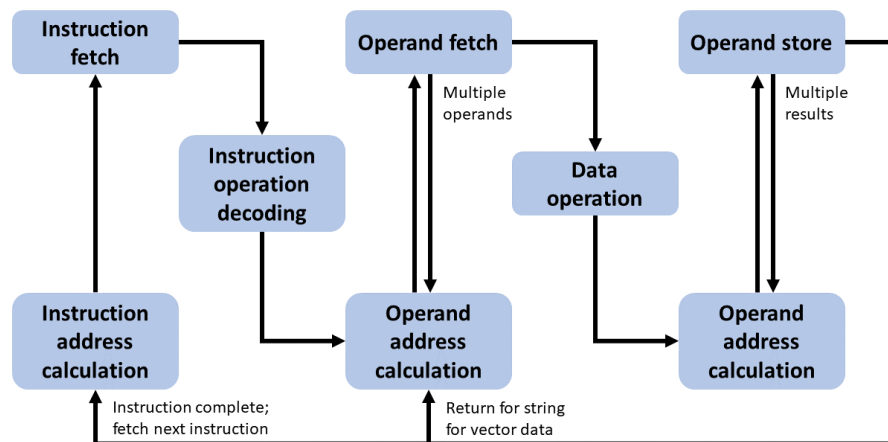


Figure 1. A diagram of instruction cycle.

In the cycle above, the address of the next instruction to be fetched could be a real address or a virtual address, depending on the architecture. Generally, the distinction is clear to the instruction set architecture. The next instruction to be fetched usually follows the current instruction. Hence, there is no explicit reference to the next instruction. On the other hand, when an explicit reference is needed, the main memory or virtual memory address must be supplied. In addition, source operands and result operands can be in the main memory,

virtual memory, processor register, I/O device, or in an immediate field in the instruction being executed (Stallings, 2019).

Operands are represented symbolically, making it possible to write machine language programs in symbolic form. Each symbolic opcode has a fixed binary representation. Programmers specifies the location of each symbolic operand. Then, a simple program would accept symbolic input, convert opcodes and operand references to binary form, and construct binary machine instructions.

A computer has a set of instructions that allow users to formulate any data processing task. Any program written in a high-level language must be translated into machine language to be executed. Thus, the set of machine instructions in a computer must be sufficient to express any instruction from a high-level language. Consequently, instructions can be categorized as follows (Stallings, 2019):

- **For data processing:** The arithmetic and logic instructions
- **For data storage:** The entry or exit of data into registers and memory locations
- **For data movement:** The input and output instructions
- **For control:** The test and branch instructions

Data Transfer

The most fundamental type of machine instruction is the data transfer instruction, which must specify the following:

- Location of the source operand (either memory, register, or top of a stack)
- Destination of the operand (either memory, register, or top of a stack)
- Length of the data to be transferred
- Addressing mode for each operand.

Common x86 instructions for data transfer:

Operation Name	Description
MOV Dest, Source	This is used to move data between registers or between registers and memory.
XCHG Op1, Op2	This swaps the contents between two registers or between register and memory.
PUSH Source	This decrements the extended stack pointer (ESP) and copies the source operand to the stack top.
POP Dest	This copies the stack top to the destination and increments the ESP.

The data transfer instructions to include in an instruction set exemplifies the kind of trade-offs the designer must make. Note that there are variants per instruction that indicate the amount of data to be transferred (8, 16, 32, or 64 bits). In addition, there are different instructions for register to register, register to memory, memory to register, and memory to memory transfers. Below are some examples of IBM ESA/390 data transfer operations (Stallings, 2019):

Operation Mnemonic	Name	Number of Bits	Description
L	Load	32	Transfer from memory to register
LH	Load Halfword	16	Transfer from memory to register
LR	Load	32	Transfer from register to register
LER	Load Short	32	Transfer from floating-point register to floating-point register
LD	Load Long	64	Transfer from memory to floating-point register
ST	Store	32	Transfer from register to memory
STH	Store Halfword	16	Transfer from register to memory
STC	Store Character	8	Transfer from register to memory
STE	Store Short	32	Transfer from floating-point register to memory

In terms of processor action, data transfer operations can be considered as the simplest type. If both source and destination are registers, then the processor simply transfers the data internally. If one or both operands are in memory, then the processor must perform some or all of the following operations (Stallings, 2019):

- Calculate the memory address based on the addressing mode
- A virtual memory address must be translated into a real memory address
- Determine whether the addressed item is in cache
- Issue a command to the memory module if the addressed item is not in cache

Data Manipulation

Conversion instructions are instructions that changes the format or operate on the format of data. An example of a conversion instruction is the IBMS ESA/390 Translate (TR) instruction that can be used to convert one 8-bit code to another. The TR instruction is composed of three operand:

TR R1 (L), R2

The operand **R2** contains the address of the start of an 8-bit code. The **L** represents the number of bytes that are translated at the specified address in **R1** (Stallings, 2019).

Input/Output (I/O) instructions encompasses the operation of transferring external data into the memory and vice versa. The format of external data of the I/O interface widely varies. Below are some examples of external representation of computer I/O data (Ledin, 2020):

- Signals on a video cable connected to a monitor
- Voltage fluctuations on ethernet cable wires
- Magnetic patterns on the surface of a disk
- Sound waves produced by speakers

Regardless of the form of the external data, the connection of any I/O device with the processor must comply with the processor's I/O instruction architecture. The processor uses the instruction categories, addressing modes, and interrupt processing methods to interact with I/O devices. The instructions read-from-location and write-to-location communicates with the I/O devices (Ledin, 2020). The following are the three (3) possible approaches in dealing with I/O operations:

1. **Programmed I/O:** The processor executes a program that provides itself a direct control over the I/O operations. These operations may include sensing the device status, sending a read or write command, and transferring of data (Stallings, 2019).
2. **Interrupt-Driven I/O:** The processor issues an I/O command, continues to execute other instructions, and is interrupted by the I/O module when the latter task is completed.
3. **Direct Memory Access (DMA):** The I/O module and the main memory exchange data directly, without the involvement of the processor. DMA is useful for transferring blocks of data within the system memory. This approach improves computer systems performance by accelerating repetitive operations (Ledin, 2020).

Common x86 instructions for input/output:

Operation Name	Description
IN Dest, Source	This copies data from the I/O port specified by the source operand to the destination operand, which is a register location.

INS Dest, Source	This copies data from the I/O port specified by the source operand to the destination operand, which is a memory location.
OUT Dest, Source	This copies the byte, word, or doubleword value from the source register to the I/O port specified by the destination operand.
OUTS Dest, Source	This copies the byte, word, or doubleword value from the source operand, which is a memory location, to the I/O port specified with the destination operand.

Control Operations

System control instructions are instructions that can only be executed while the processor is in a certain privileged state, or is executing a program in a distinct privileged area of the memory. These instructions are typically reserved for the use of the operating system. Some examples of system control operations are as follows (Stallings, 2019):

- Reading or altering a control register
- Reading or modifying a storage protection key
- Accessing process control blocks in a multiprogramming system

Significant fraction of instructions in any program encompasses function that changes the sequence of instruction execution. Hence, the implementation of **transfer of control instructions**. For these instructions, the operation performed by the processor is to update the program counter that will contain the address of a particular instruction in memory. Below are some reasons why transfer of control operations are required (Stallings, 2019):

- It is essential to be able to execute each instruction more than once
- Virtually all programs involve decision making
- It helps mechanisms for breaking up tasks into smaller pieces that can be worked on one at a time

Common x86 instructions for transfer of control:

Operation Name	Description
CALL Proc	This saves the procedure linking information on the stack and branches to the called procedure that is specified by the operand.
RET	This transfers the program control to a return address located on the top of the stack. The return is made to the instruction that follows the CALL instruction.
JMP Dest	This transfers program control to a different point in the instruction stream without recording the return information.

	The operand specifies the address of the instruction being jumped to.
Jcc Dest	This checks the state of one or more status flags in the register. If the flags are in the specified state or condition, a JMP operation to the target instruction specified by the operand is performed.
NOP	This is a one-byte or multi-byte instruction that takes up space in the instruction stream but does not impact the machine context. This instruction performs no operation.
HLT	This stops instruction execution and places the processor in a halt state.
WAIT	This causes the processor to repeatedly check for and handle pending, unmasked, floating-point exceptions before proceeding.
INT Nr	This interrupts the current program and runs a specified interrupt program.

Intel x86 and ARM Instruction Set

Intel x86 instruction set involves complex array of operation types, including some specialized instructions. This instruction set intended to provide tools for the compiler to produce optimized machine language translations from high-level language programs. Intel x86 encompasses conventional instructions that are found in most machine instruction set and several types of tailored instructions.

- **Procedural instructions:** The Intel x86 provides four (4) instructions that supports procedural call/return which are: CALL, ENTER, LEAVE, and RETURN.
- **Memory management instructions:** These are considered specialized instructions that deals with memory segmentation. These instructions can only be executed from the operating system. They allow local and global segment tables to be loaded and read, and for the privilege level of a segment to be checked and altered.
- **Status flags and condition codes instructions:** In Intel x86 status flags are set through arithmetic and compare operations. The compare operation in most languages subtracts two (2) operands and the result sets the status flags.
 - **Status flags** – These are bits in a special register that may can be set by certain operations and can be used in conditional branch instructions.

- **Condition codes** – These refer to the setting of one (1) or more status flags. Below are some examples of Intel x86 condition codes (Stallings, 2019):

Symbol	Condition Tested	Comment
A, NBE	C=0 AND Z=0	Above; Not below or equal (greater than, unsigned)
B, NAE, C	C=1	Below; Not above or equal (less than, unsigned); Carry set
NP, PO	P=0	No parity; Parity odd
S	S=1	Sign (negative)

The **Advanced RISC Machines (ARM) instruction set** encompasses a large collection of operation types that may fall under any of the principal categories below:

- **Load and store instructions:** In the ARM instruction set, only the load and the store instructions can access memory locations. The arithmetic and logical instructions are only performed on registers and immediate values encoded in the instruction.
- **Branch instructions:** The ARM instruction set supports a branch instruction that allows a conditional branch forward or backward up to 32MB. A subroutine call can also be performed by a variant of the standard branch instruction.
- **Data processing instructions:** This involves the logical instructions AND, OR, XOR, add and subtract instructions, and the test and compare instructions.
- **Multiply instructions:** The multiply instructions operate on word or halfword operands and can produce normal or long results.
- **Parallel addition and subtraction instructions:** Portions of two (2) operands are operated in parallel. These instructions are useful in image processing applications.
- **Extended instructions:** These are instructions for unpacking data by sign or zero, and extending bytes to halfwords or words, and halfword to words.
- **Status register access instructions:** The ARM instruction set provides the ability to read and write portions of the status register.

The ARM instruction set relies heavily on register addressing since it is based on the design of reduced instruction set computers. The utilization of conditional execution and conditional setting of the conditional flags in ARM helps in developing shorter programs that use less memory. On the other hand, all instructions in ARM include a condition code field of 4 bits, and all

data processing instructions (arithmetic and logic) include an S bit that signifies whether the instruction updates the condition flags.

References:

- Ledin, J. (2020). *Modern computer architecture and organization*. Packt Publishing
- Stallings, W. (2019). *Computer organization and architecture: Designing for performance* (11th ed.). Pearson Education, Inc.